AI Agent Developer Intern — Technical Assignment

Vilambo Private Limited

- **Submission Deadline: 24 July 2026, 11:59 PM IST**
- This assignment is **mandatory** for interview shortlisting
- Video demo (max 3 minutes) **+** GitHub code repository required
- Optional: Build a UI to showcase full-stack skills

Assignment Overview

Build an **AI-Powered Research Paper Analyzer** that uses multiple specialized AI agents to automatically read, analyze, and summarize academic research papers. This assignment tests your ability to design, orchestrate, and implement production-grade multi-agent systems with quality control and iterative improvement.

Problem Statement

Researchers, students, and professionals often need to quickly understand academic papers to decide if they're relevant to their work. Reading full papers is time-consuming, and manually extracting key information (methodology, findings, citations) is tedious and error-prone.

Your task is to build an intelligent multi-agent system that automates this process by:

- Analyzing the research paper to extract methodology, key findings, and core concepts
- Generating a clear, concise executive summary of the paper
- Extracting and organizing all citations and references
- Identifying key insights and practical takeaways
- Ensuring quality through an automated review and iteration process
- Combining all outputs into a comprehensive research brief

Technical Requirements

1. Framework Choice (Choose ONE)

You must implement this using **one** of the following frameworks:

- **Option A: LangGraph** — Build a state-based multi-agent workflow with nodes, edges, and conditional routing
- **Option B: CrewAI** — Design a crew with specialized agents, defined roles, and task delegation
- **Option C: n8n** — Create a no-code workflow with LLM nodes, function nodes, and conditional logic

2. Multi-Agent Architecture (REQUIRED)

Your system must include the following agent types:

Boss Agent (Orchestrator)

- Coordinates the entire research analysis workflow
- Delegates tasks to specialized sub-agents
- Monitors progress and handles failures
- Combines final outputs into a complete research brief

Sub-Agents (At least 3 required)

- **Agent 1 — Paper Analyzer:** Extracts research methodology, hypothesis, experiments, and key findings
- **Agent 2 — Summary Generator:** Creates a clear executive summary (150–200 words) covering problem, approach, and results
- **Agent 3 — Citation Extractor:** Identifies and organizes all citations, references, and key papers mentioned
- **Agent 4 — Key Insights (Bonus):** Generates actionable takeaways and practical implications (optional but recommended)

Review Agent (Quality Control)

- Reviews each sub-agent's output for accuracy, completeness, and clarity
- Provides feedback and quality scores (e.g. 1–10 scale)
- Triggers re-generation if quality is below threshold (e.g. score < 7)
- Limits iterations (e.g. max 2 retries per agent to prevent infinite loops)
- Validates that extracted information matches the source paper

3. Workflow Requirements

1. **Input:** Research paper PDF or text (URL to PDF or uploaded file)
2. Boss Agent receives input and delegates to Paper Analyzer Agent
3. Paper Analyzer extracts methodology and findings -> passed to Review Agent

4. If approved -> Boss Agent delegates to Summary Generator, Citation Extractor, and Key Insights agents (parallel or sequential)
5. Each output is reviewed -> iterated if below threshold -> approved when quality is sufficient
6. Boss Agent combines all approved outputs into final research brief
7. **Output:** Complete research brief with analysis, summary, citations, and key insights

4. LLM Integration

- Use OpenAI API (GPT-4o-mini recommended for cost) **OR** Anthropic Claude API **OR** Google Gemini API
- Implement prompt engineering for each agent with clear, specific instructions
- Use structured outputs (JSON mode or Pydantic models) for agent responses
- Handle API errors gracefully (retry logic, fallback strategies, rate limiting)
- Manage context windows effectively (research papers can be long)

5. PDF Processing (Important)

- Extract text from PDF files (use `PyPDF2`, `pdfplumber`, or similar libraries)
- Handle multi-page papers and preserve document structure
- Alternative: Accept text input if PDF parsing is too complex
- Bonus: Handle different paper formats (arXiv, IEEE, academic journals)

6. Code Quality Expectations

- Clean, readable, and well-commented code
- Proper error handling and logging throughout the workflow
- Environment variables for API keys (**NEVER** hardcode secrets)
- `README.md` with setup instructions, architecture diagram, and usage examples
- `requirements.txt` (Python) or `package.json` (JavaScript) for dependencies

Deliverables

1. Demo Video (REQUIRED — Max 3 Minutes)

- Show the system analyzing a real research paper (use a public arXiv paper or academic PDF)
- Explain the multi-agent workflow at a high level (30–45 seconds)
- Walk through the code structure and key components (~1 minute)
- Show the final output: research brief with analysis, summary, citations, and insights
- Upload to Google Drive and make the link **PUBLIC** (anyone with the link can view)

2. GitHub Repository (REQUIRED)

- Public GitHub repo with all source code
- `README.md` with clear setup instructions, architecture explanation, and usage guide
- Include an architecture diagram (flowchart or visual representation of agent workflow)
- Sample input (example research paper) and output (generated research brief)
- Document any limitations or known issues

3. Optional: User Interface (Bonus Points)

- Build a simple web UI using React, Next.js, Streamlit, or Gradio
- Allow users to upload a PDF or paste a paper URL and see the generated research brief
- Display agent workflow progress visually (e.g. which agent is currently running)
- Show review scores and iteration history for transparency
- Deploy the UI (Vercel, Streamlit Cloud, Render, or any free hosting) and share the live link

Evaluation Criteria

We will evaluate your submission based on the following:

Criteria	Weight	What we look for
Architecture	30%	Multi-agent orchestration (boss + sub-agents + review + workflow logic)
LLM Integration	25%	Prompt quality, context engineering, structured outputs, error handling
Review & Iteration	20%	Review agent, quality scoring, iteration logic, threshold management
Code Quality	15%	Readability, error handling, documentation, dependency management
Presentation	10%	Video clarity, demo quality, explanation of decisions
Bonus	Extra	UI, deployment, PDF parsing, features beyond requirements

Submission Instructions

Submit your work using **your personal submission link**, sent to you in the invitation email for this

assignment (not through this PDF). That page will ask you for:

- GitHub repository link (**required**, must be public)
- Demo video link (**required** — upload to Google Drive and make it public, "Anyone with the link can view")
- Live UI link (**optional**, if you built and deployed one)

> ***Ensure all links are PUBLIC*** *and accessible — test them in incognito mode. Do* ***NOT*** *hardcode API keys in your GitHub repo — use* `.env` *files and* `.gitignore`*. Submissions after* ***24 July 2026*** *will not be considered. Video file should be under 100MB — compress if needed.*

Tips for Success

- Start with a simple workflow, then add complexity incrementally
- Test each agent independently before connecting them in the full workflow
- Use smaller LLM models (GPT-4o-mini, Claude Haiku, Gemini Flash) to reduce API costs during testing
- Implement detailed logging to debug agent interactions and track workflow state
- Keep your video concise and focused — show **WHAT** it does, **HOW** it works, and **WHY** you designed it that way
- Test with multiple research papers to ensure your system generalizes well
- Make your README clear and comprehensive — we should be able to run your code by following your instructions
- If PDF parsing is challenging, start with text input and add PDF support later

Sample Input & Expected Output

Sample Input (Research Paper). You can use any publicly available research paper. Good sources: arXiv.org, Papers with Code, Google Scholar, or any academic paper you have access to.

Expected Output (Research Brief). Your system should generate a structured research brief containing:

1. **Paper Metadata** — title, authors, publication year, venue
2. **Research Analysis** — problem statement, methodology, key experiments, main findings
3. **Executive Summary** — concise 150–200 word overview
4. **Citations & References** — all papers cited and key related work
5. **Key Insights (Bonus)** — practical takeaways, implications, potential applications

Sample Architecture Diagram

Your agent workflow might look something like this:

```
Input (Research Paper PDF/Text)
    |
Boss Agent (Orchestrator)
    |
Paper Analyzer Agent  -> Review Agent -> [Approve / Reject & Iterate]
    | (if approved)
Summary Generator     -> Review Agent -> [Iterate if score < 7]
Citation Extractor    -> Review Agent -> [Iterate if score < 7]
Key Insights Agent    -> Review Agent -> [Iterate if score < 7]
    | (all approved)
Boss Agent combines all outputs
    |
Output (Complete Research Brief)
```

Technical Hints

For LangGraph users

- Define a state schema with fields for `paper_text`, `analysis`, `summary`, `citations`, `insights`, `review_scores`
- Create nodes for each agent (analyzer, summarizer, citation_extractor, reviewer, combiner)
- Use conditional edges to route based on review scores (if score >= 7, proceed; else retry)
- Implement a retry counter in state to prevent infinite loops

For CrewAI users

- Define roles for each agent (Paper Analyst, Summarizer, Citation Expert, Quality Reviewer)
- Set clear goals for each agent's task
- Use a sequential or hierarchical process depending on your workflow design
- Implement custom validation in the review agent to trigger task retries

For n8n users

- Use Function nodes to process and validate responses
- Use IF nodes for conditional routing based on review scores
- Use Set nodes to accumulate results from different agents
- Add error-handling nodes to catch API failures

- **Deadline: 24 July 2026, 11:59 PM IST**
- **Questions?** Email us at operations@vilambo.com
- Only candidates who submit this assignment will be shortlisted for interviews

Good luck! We're excited to see what you build.

Vilambo Private Limited · Building Intelligent Products that Empower People